

Beyti V2.1 - Database Design Evolution

Overview

During the development of the Beyti e-commerce platform, several critical design gaps were identified in the initial database schema (V1). This document outlines the evolution to V2.1, explaining the reasoning behind each modification and how they improve the system's functionality and maintainability.

Initial Problems Identified

The original schema had some architectural inconsistencies that needed addressing:

- Inconsistent Role Modeling:** While `Seller`, `ServiceProvider`, and `Driver` had dedicated tables linked to `UserProfile`, customers were only represented by a `RoleType` field. This created an imbalance in how different user types were handled.
- Missing Address Infrastructure:** There was no proper way to store addresses for deliveries, shop locations, or service appointments. Orders referenced "CustomerId" but had no delivery address fields, and drivers had no way to access location information.
- Incomplete Service Provider System:** Service providers existed in the schema but lacked a booking mechanism. There was no way for customers to actually schedule and pay for services.
- Payment System Limitations:** The payment table only handled product orders, with no consideration for service payments or deposits.

Major Changes Implemented

1. Customer Table Addition

What Changed: Created a new `Customer` table following the same pattern as other role tables.

```
sql

CREATE TABLE dbo.Customer(
  Id          INT IDENTITY(1,1) PRIMARY KEY,
  UserProfileId INT NOT NULL UNIQUE,
  Phone       NVARCHAR(30) NOT NULL,
  CreatedAt   DATETIME2(3),
  UpdatedAt   DATETIME2(3)
);
```

Why This Matters:

- Provides architectural consistency across all user roles

- Gives customers a proper anchor for relationships (addresses, orders, reviews)
- Allows customer-specific data without polluting the base `UserProfile` table
- Makes the codebase more maintainable by following a single pattern

Impact on Orders: The `Orders` table was updated from referencing `CustomerUserId` (pointing to `UserProfile`) to `CustomerId` (pointing to `Customer`), making the relationship clearer and more consistent.

2. Centralized Address System

What Changed: Implemented a central `Address` table with junction tables for each entity type that needs addresses.

Design Decision: Rather than creating separate address tables for each entity (`CustomerAddress`, `SellerAddress`, etc.), we use one `Address` table with multiple junction tables. This approach offers:

- **Reusability:** The same address validation and geocoding logic works everywhere
- **Simplicity:** One table to maintain, one set of indexes to optimize
- **Flexibility:** Easy to add new entity types that need addresses later

Key Fields in Address:

```
sql
- Street, City, Region, PostalCode, Country (location details)
- Latitude, Longitude (for driver navigation and distance calculations)
- Label (e.g., "Home", "Work", "Main Shop")
- IsDefault (marks user's primary address)
- IsActive (soft delete capability)
```

Junction Tables:

- `CustomerAddress`: Links customers to their saved addresses (one-to-many)
- `SellerAddress`: Links sellers to shop locations (supports multiple branches)
- `ServiceProviderAddress`: Links providers to their business locations

Why Multiple Addresses Per Entity: Customers might have a home address, work address, and parents' house. Sellers could operate multiple storefronts. This design accommodates growth without schema changes.

3. Phone Number Consolidation

Initial Consideration: We debated whether phone numbers should be stored in the `Address` table (different contact per location) or in the user role tables (one phone per person).

Final Decision: Phone numbers are stored at the role level ((Customer.Phone), (Seller.Phone), (ServiceProvider.Phone), (Driver.Phone)) rather than per-address.

Reasoning:

- Simplifies contact during deliveries and services
- In Bahrain's context, most users have one primary mobile number
- Reduces redundancy and update complexity
- Still allows future extension if per-location contacts become necessary

4. Service Booking System

What Changed: Added a complete service booking workflow with scheduling, custom pricing, and split payments.

New Tables:

TimeSlot Table: Manages service provider availability using a recurring weekly schedule.

```
sql

CREATE TABLE dbo.TimeSlot(
    ServiceProviderId INT,
    DayOfWeek        TINYINT, -- 0-6 (Sunday-Saturday)
    StartTime         TIME,
    EndTime           TIME,
    IsActive          BIT
);
```

Design Approach: Providers set their weekly availability (e.g., "Every Monday 9 AM - 5 PM, Every Wednesday 2 PM - 6 PM"). The application layer generates specific bookable slots for the next 30 days. This balance between database simplicity and application flexibility avoids storing thousands of individual time slots.

ServiceBooking Table: Handles the complete service appointment lifecycle.

Key Features:

- **Custom Quotes:** Providers can set custom prices within their min/max range
- **Deposit System:** Mandatory 50% upfront payment, 50% upon completion
- **Cancellation Tracking:** Records who canceled, when, and why
- **Address Integration:** Links to customer's service location for home services

Status Flow:

```
PendingQuote → QuoteSent → DepositPaid → Confirmed →  
InProgress → AwaitingFinalPayment → Completed / Canceled
```

Cancellation Rules Enforced:

- Customers can cancel freely before provider sends a quote
- After deposit is paid but before confirmation: customer loses deposit
- After confirmation: cancellation tracked with potential fees
- All tracked via `CanceledBy`, `CancellationReason`, and `CancellationFee` fields

5. ServiceCatalog Enhancements

What Changed: Added pricing guidance and time estimates to predefined services.

```
sql
```

```
ALTER TABLE ServiceCatalog ADD  
  MinPrice      DECIMAL(10,2),  
  MaxPrice      DECIMAL(10,2),  
  EstimatedDuration INT; -- in minutes
```

Purpose:

- Sets customer expectations about pricing
- Helps providers quote within reasonable ranges
- Assists in scheduling by estimating appointment length
- Provides data for future analytics on service pricing trends

6. Unified Payment System

What Changed: Extended the `Payment` table to handle both product orders and service bookings rather than creating a separate `ServicePayment` table.

Implementation:

```
sql
```

ALTER TABLE Payment ADD

```
PaymentFor    NVARCHAR(20), -- 'Order' or 'ServiceBooking'  
ServiceBookingId INT,  
PaymentType   NVARCHAR(20); -- 'Deposit', 'Final', 'Full'
```

Check Constraint: Ensures that exactly one of `OrderId` or `ServiceBookingId` is populated based on `PaymentFor` type.

Advantages of Unified Approach:

- Single integration with payment gateways (Benefit, BenefitPay, etc.)
- Consistent refund and dispute handling logic
- Unified payment history for users across products and services
- Simpler audit trails and financial reporting

Trade-off Acknowledged: If service payments become significantly more complex (installments, tips, bonuses), we may need to split this later. For MVP, the unified approach reduces code duplication and complexity.

7. Delivery System Updates

What Changed: `DeliveryTicket` now references both pickup and delivery addresses explicitly.

sql

ALTER TABLE DeliveryTicket ADD

```
PickupAddressId INT, -- Seller's shop  
DeliveryAddressId INT; -- Customer's location
```

Driver Access Pattern: Created `vw_DeliveryRoutes` view that provides drivers with:

- Complete route information (pickup → delivery)
- GPS coordinates for navigation integration
- Contact phone numbers for both seller and customer
- Order details and payment status

Why This Structure: Separating pickup and delivery addresses in the ticket (rather than just linking to Order) allows the system to handle edge cases like "customer changed delivery address after order" or "driver picks up from warehouse instead of shop."

8. Service Review System

What Changed: Created a separate `ServiceReview` table instead of extending the product `Review` table.

Comparison:

Aspect	Product Review	Service Review
References	OrderId, ProductId	ServiceBookingId, ServiceProviderId
Rating Fields	Single overall rating	Overall + Quality + Professionalism + Timeliness
Response	Sellers can hide comments	Providers can respond directly
Context	Physical product quality	Service experience

Design Rationale: While we could have used a discriminator pattern (one table for both), services and products are fundamentally different review contexts. Services benefit from multi-dimensional ratings (was the provider professional? were they on time?), while products focus on quality and description accuracy.

Keeping them separate:

- Makes queries simpler and faster
- Allows service-specific features without affecting product reviews
- Provides clearer separation in the application layer
- Easier to add service-specific fields later (e.g., "would you book again?")

Orders Table Integration

What Changed: Added address references to support different fulfillment types.

```
sql
```

```
ALTER TABLE Orders ADD
  CustomerId INT,
  DeliveryAddressId INT, -- Used when FulfillmentType = 'Delivery'
  PickupAddressId INT; -- Seller's shop when FulfillmentType = 'Pickup'
```

Workflow:

- **Pickup Orders:** `PickupAddressId` references the seller's shop location
- **Delivery Orders:** Both `PickupAddressId` (where driver collects) and `DeliveryAddressId` (where driver delivers) are populated
- When delivery is created, `DeliveryTicket` copies these addresses for driver access

Helper Views for Application Logic

Rather than making the application repeatedly join multiple tables, we created optimized views for common query patterns:

vw_DeliveryRoutes: Provides drivers with everything needed for a delivery in one query - addresses, contacts, GPS coordinates, and order details.

vw_ServiceProviderBookings: Shows service providers their upcoming appointments with full customer address details (filtered for home services only).

vw_CustomerOrderHistory: Gives customers a clean view of their order history with relevant address information.

vw_ProviderAvailability: Displays available time slots for booking forms, showing which providers are available on which days.

Data Integrity Measures

Several constraints and checks ensure data consistency:

Check Constraints:

- `ServiceCatalog` and `ServiceProvider` verify $\text{MinPrice} \leq \text{MaxPrice}$
- Ratings constrained between 1-5
- Stock quantities can't go negative
- `TimeSlot` ensures $\text{StartTime} < \text{EndTime}$
- Payment table ensures only one of `OrderId` or `ServiceBookingId` is set

Computed Columns:

- `ServiceBooking.DepositAmount` and `FinalAmount` auto-calculate from `QuotedPrice`
- `Orders.TotalAmount` sums subtotal and delivery fee
- `OrderItem.LineTotal` multiplies quantity and unit price

Soft Deletes: The `IsActive` field on `Address` and other tables allows logical deletion without losing referential integrity or historical data.

Performance Considerations

Indexing Strategy:

- Foreign keys have corresponding indexes for join performance

- `IsActive` flags included in composite indexes for filtered queries
- `(RecipientUserId, IsRead)` on notifications for fast unread counts
- `(ServiceProviderId, DayOfWeek, IsActive)` on TimeSlot for availability lookups

View Performance: Views are not materialized, allowing them to always show current data. For performance-critical queries, these could be converted to indexed views if testing shows the need.

Migration Path from V1 to V2.1

For existing installations moving from V1, the key steps would be:

1. Create new tables (Customer, Address, ServiceBooking, etc.)
2. Migrate existing order data to create Customer records
3. Prompt users to add address information on next login
4. Update Orders.CustomerId from the new Customer.Id values
5. Deploy updated application code that uses new relationships
6. After verification, drop old CustomerUserId column from Orders

Scalability Considerations

What Works at Scale:

- Central address table reduces data duplication
- Junction tables allow unlimited addresses per entity
- View-based access patterns separate read concerns from write concerns
- Indexed queries on Status fields support efficient filtering

Future Optimization Opportunities:

- TimeSlot generation could move to a materialized table if booking volume grows
- Address geocoding could be cached in a separate table
- Notification table might benefit from partitioning by date
- Payment transaction history could archive to separate table after 1 year

Conclusion

The evolution from V1 to V2.1 transformed the schema from a product-focused e-commerce database to a comprehensive multi-sided marketplace supporting both products and services. The key improvements—

consistent role modeling, centralized address management, unified payments, and complete service workflows —provide a solid foundation for the Beyti platform's MVP and future growth.

The design balances immediate needs with future flexibility, using proven patterns (junction tables, soft deletes, computed columns) while avoiding over-engineering. Each decision documented here represents a trade-off between simplicity and capability, chosen to deliver working functionality quickly while maintaining a maintainable codebase.